TRABAJO FINAL PAMN

Desarrollo de una aplicación móvil nativa

El trabajo final consiste en diseñar, desarrollar, documentar y defender una aplicación móvil nativa en equipos de dos o tres estudiantes. La aplicación se desarrollará en Kotlin para Android o en Swift para iOS. Se entregarán el proyecto en GitHub y una memoria en PDF, y se realizará una demostración con defensa técnica.

Objetivo

Aplicar los conocimientos de la asignatura al ciclo completo de desarrollo de una aplicación: análisis de requisitos, diseño de la interfaz, elección de arquitectura, implementación, pruebas y documentación. El equipo deberá justificar sus decisiones y demostrar que comprende el funcionamiento del proyecto entregado.

Enunciado

La temática es libre y puede incluir aplicaciones de gestión, ocio, educación u otros ámbitos, así como juegos. El alcance deberá ser suficiente para evidenciar los conocimientos adquiridos y viable dentro del tiempo disponible. En los equipos de tres se esperará un alcance o una profundidad técnica acordes con el mayor número de integrantes; añadir pantallas o tecnologías sin utilidad no incrementará la valoración.

Requisitos del proyecto

Aplicación funcional

Implementar las funcionalidades principales definidas en los requisitos y ofrecer un recorrido completo de uso.

Interfaz y navegación

Incluir varias pantallas o vistas con navegación coherente. En un juego pueden corresponder, por ejemplo, al menú, la partida y los resultados. Cuidar consistencia, legibilidad y respuesta a las acciones del usuario.

Organización técnica

Aplicar y justificar una arquitectura que separe responsabilidades. Mantener una estructura clara del código, gestionar el estado y tratar los errores y estados de carga cuando proceda.

Datos y ejecución

Elegir y justificar cómo se gestionan los datos. Cuando se necesite persistencia, podrá ser local, remota o combinada. Las operaciones costosas o de entrada y salida deberán realizarse sin bloquear la interfaz, utilizando mecanismos adecuados a la plataforma.

Verificación

Realizar y documentar pruebas de las funcionalidades principales y de situaciones de error relevantes. Los tests automatizados son recomendables, pero no se establece una cobertura mínima.

La autenticación, las bases de datos, las APIs externas y las funciones del dispositivo —cámara, geolocalización, sensores o notificaciones— se elegirán según las necesidades del proyecto. Las APIs y las funciones del dispositivo no son obligatorias. Se recomienda emplear servicios como Firebase o Supabase. Si no se requiere autenticación, base de datos o persistencia, se deberá justificar la decisión.

Proceso de desarrollo y repositorio GitHub

GitHub deberá utilizarse durante el desarrollo para dejar evidencia del proceso seguido. No será suficiente subir el proyecto completo al final mediante uno o unos pocos commits. La evaluación atenderá a la calidad de las evidencias y a la contribución real de los integrantes, sin basarse únicamente en el número de commits.

Organización del equipo

Identificar a los integrantes y explicar el reparto inicial de tareas, los cambios realizados y la contribución de cada persona. Todos deberán participar en el desarrollo y comprender el funcionamiento general de la aplicación.

Planificación y seguimiento

Utilizar issues para registrar funcionalidades, tareas e incidencias, y organizar las etapas mediante hitos o un tablero. Como referencia, distinguir análisis y diseño, implementación, pruebas y preparación de la entrega.

Evolución e integración

Realizar commits progresivos con mensajes descriptivos. Utilizar ramas y pull requests para integrar y revisar cambios de forma razonable en el trabajo en equipo. Relacionar las tareas con los cambios cuando sea posible y mantener una versión estable identificable.

Documentación del repositorio

El README deberá incluir descripción y objetivos, autores, funcionalidades, capturas de pantalla, tecnologías, resumen de la arquitectura e instrucciones de configuración, compilación y ejecución. Indicar versiones de las herramientas, dependencias y requisitos necesarios para reproducir el proyecto.

Servicios y recursos

Documentar los servicios utilizados y la configuración necesaria, proporcionando ejemplos sin credenciales reales. No incluir contraseñas ni claves privadas. Identificar librerías, recursos gráficos, fragmentos reutilizados y sus fuentes o licencias.

Entrega

La entrega se realizará en la actividad correspondiente del Aula Virtual. Se debe entregar lo siguiente:

• Enlace al repositorio de GitHub, accesible al profesorado, con el código completo y la documentación necesaria para ejecutar la aplicación.

• Memoria en PDF con los diez apartados indicados en este documento, nombre de la aplicación, integrantes y enlace al repositorio.

• Identificación inequívoca de la versión entregada mediante una etiqueta o release y el commit correspondiente.

• Instrucciones y recursos para la demostración y las pruebas.

No se exige publicar en Google Play ni en App Store. Para Android puede adjuntarse un APK de demostración. En iOS deberá explicarse cómo compilar y ejecutar el proyecto en el entorno disponible; la distribución mediante TestFlight no es obligatoria.

Estructura de la memoria

La memoria debe explicar el proyecto realizado y sus decisiones con precisión. Se incluirán diagramas, capturas y fragmentos de código cuando ayuden a comprenderlo, evitando reproducir el código completo. Los apartados no aplicables deberán justificarse. Se añadirá al final una relación de las fuentes consultadas.

1 Introducción y objetivos

Describir la aplicación, la necesidad o experiencia que ofrece, el público objetivo y los objetivos. Delimitar el alcance y las funcionalidades principales.

2 Análisis de requisitos

Presentar requisitos funcionales y no funcionales identificados y verificables. Incluir actores, historias de usuario o casos de uso, si lo consideran necesario, cuando ayuden a explicar el comportamiento.

3 Diseño de la interfaz y experiencia de usuario

Mostrar las pantallas, su navegación y los prototipos iniciales. Justificar las decisiones de interacción y explicar los cambios respecto al diseño inicial. Considerar legibilidad, accesibilidad básica y adaptación a los dispositivos previstos.

4 Arquitectura de la aplicación

Identificar y justificar la arquitectura adoptada. Incluir un diagrama de componentes o capas, sus responsabilidades y su comunicación, relacionándolo con la implementación real.

5 Gestión y persistencia de datos

Explicar los datos y su modelo, relaciones y mecanismos de almacenamiento. Si existe base de datos, documentar su diseño. Si no es necesaria, justificarlo y describir la gestión del estado y de otros datos relevantes.

6 Desarrollo e implementación

Explicar la estructura de carpetas, ficheros y componentes principales, la gestión del estado, los patrones y las librerías. Describir la programación asíncrona utilizada —por ejemplo, corrutinas en Kotlin o async/await en Swift— Documentar servicios, APIs o funciones del dispositivo y las decisiones técnicas relevantes.

7 Pruebas y validación

Registrar casos de prueba con condiciones, acciones, resultado esperado y resultado obtenido. Identificar dispositivos o emuladores y versiones probadas, errores detectados y correcciones. Aportar evidencias de las funcionalidades principales y de errores relevantes; describir tests automatizados si los hay.

8 Proceso de desarrollo y gestión del proyecto

Explicar etapas, organización del equipo, distribución de tareas y cambios de planificación. Incluir enlaces a issues, hitos, ramas y pull requests representativos y una síntesis de las aportaciones individuales.

9 Uso de herramientas de IA generativa

Declarar si se ha utilizado IA. En caso afirmativo, identificar herramientas, finalidad, partes del proyecto afectadas y ejemplos representativos de instrucciones y resultados. Explicar cómo se revisaron, adaptaron y comprobaron las aportaciones.

10 Conclusiones y trabajo futuro

Valorar el cumplimiento de los objetivos con referencia a los resultados. Explicar dificultades, aprendizajes, limitaciones y mejoras pendientes, diferenciándolas de las funcionalidades entregadas.

Presentación y defensa

La sesión tendrá una duración orientativa de 12–13 minutos por equipo, incluidas las preguntas. Se propone dedicar unos 8 minutos a la presentación y demostración, y 4–5 minutos a la defensa técnica. Todos los componentes del grupo deberán intervenir.

Presentación del proyecto

Explicar brevemente la finalidad, el público objetivo y el alcance. Exponer las decisiones principales de arquitectura, interfaz y datos, evitando convertir la presentación en una lectura de la memoria.

Demostración de la aplicación

Mostrar un recorrido funcional representativo y alguna situación de error o caso límite relevante. Preparar previamente el dispositivo o emulador y los datos de prueba.

Defensa técnica

El profesorado podrá preguntar a cualquiera de los integrantes sobre requisitos, arquitectura, código, datos, pruebas, contribución personal y uso de IA. Cada estudiante deberá explicar las decisiones del proyecto y el código de las partes en las que ha trabajado, además de comprender el funcionamiento general.

Uso de IA generativa

Se permite utilizar herramientas de IA generativa como apoyo al análisis, diseño, programación, depuración, pruebas o documentación. Su uso deberá declararse expresamente, de acuerdo con el PDA, y documentarse en el apartado correspondiente de la memoria.

El equipo es responsable de la corrección del resultado, también cuando parte del código o del texto proceda de una herramienta de IA. Deberá verificar las sugerencias, explicar las modificaciones realizadas y comprobar su integración mediante pruebas. La entrega de código que los integrantes no comprendan impedirá acreditar las competencias correspondientes en la defensa.

Si no se han utilizado estas herramientas, bastará con indicarlo expresamente. Utilizarlas o no utilizarlas no otorga puntos por sí mismo: se valorarán la transparencia, la revisión crítica y el dominio del trabajo entregado.

Indicaciones para preparar el trabajo

Elegir un alcance asumible y priorizar un recorrido funcional completo. Mantener la memoria y los requisitos alineados con la evolución de la aplicación. Registrar las decisiones y pruebas durante el desarrollo para que la documentación final refleje el proceso real.

La incorporación de autenticación, servicios externos o funciones móviles podrá mejorar la calidad técnica cuando resuelva una necesidad y esté bien integrada. No se penalizará su ausencia si no son pertinentes y la aplicación demuestra una complejidad suficiente. El funcionamiento sin conexión, la accesibilidad y los tests automatizados también se valorarán dentro de los criterios correspondientes.

Criterios de evaluación

El trabajo se calificará sobre 10 puntos mediante la siguiente rúbrica. Los criterios se aplicarán según la naturaleza y el alcance del proyecto, sin exigir tecnologías que no sean pertinentes. La valoración se apoyará en la aplicación, el repositorio, la memoria y la defensa.

Criterio	Nota	Evidencias y aspectos evaluados
Funcionalidad y validación	3,0	Cumplimiento de requisitos, recorrido completo, estabilidad, tratamiento de errores y pruebas con resultados y evidencias.
Arquitectura y calidad técnica	2,0	Separación de responsabilidades, organización y legibilidad del código, gestión de estado y datos, ejecución asíncrona cuando proceda y decisiones justificadas.
Proceso de desarrollo y GitHub	1,5	Evolución progresiva, planificación, ramas y revisión de cambios, reproducibilidad y aportaciones documentadas de todos los integrantes.
Memoria y documentación	1,5	Claridad y coherencia con el producto, explicación de decisiones, diagramas y evidencias útiles, fuentes y declaración del uso de IA.
Interfaz y experiencia de usuario	1,0	Navegación, consistencia, legibilidad, adaptación al dispositivo, accesibilidad básica y respuesta a las acciones del usuario.
Presentación y defensa	1,0	Demostración eficaz, ajuste al tiempo, participación de todos los integrantes y dominio técnico demostrado en las respuestas.

Total máximo: 10 puntos.

Niveles de logro

En cada criterio se asignará entre el 0 % y el 100 % de su puntuación máxima. Las siguientes referencias permiten interpretar el grado de cumplimiento; se podrán utilizar puntuaciones intermedias.

Excelente — 100 %: cumplimiento completo, decisiones justificadas y evidencias sólidas, sin carencias relevantes.

Adecuado — 75 %: cumplimiento sustancial y evidencias suficientes, con limitaciones menores.

Básico — 50 %: cumplimiento parcial, con carencias relevantes o evidencias limitadas.

Insuficiente — 25 %: cumplimiento escaso, errores importantes o justificación muy débil.

No acreditado — 0 %: ausencia de la evidencia o imposibilidad de demostrar el criterio.

La contribución y el dominio individual se comprobarán mediante las evidencias del repositorio, la memoria y las respuestas durante la defensa. La calificación no se determinará por un recuento de commits.

Relación con el PDA

Esta rúbrica determina la nota del trabajo final. Su peso en la asignatura y las condiciones de superación se regirán por el PDA vigente: 50 % en evaluación continua y 30 % en evaluación no continua. Para aplicar la ponderación, tanto SE1 como SE2 deben alcanzar al menos 5 puntos, conforme al PDA.

Referencia: PDA de PAMN 2026–2027, accesible desde la web de la asignatura: https://mltechdrawer.github.io/PAMN/